SIMATS ENGINEERING

Engineer to Excel

SIMATS Online Programming Lab

JAVA PROGRAMMING

M MOSHIKA
192424064RA

CO5-AT2-Application Based Problem Solving

← Back

QUESTIONS

Q1
Question 4 — Smart Home Automation Controller
100 marks

1/1 answered

Q1 Question 4 — Smart Home Automation Controller

100 marks

Design an abstract class Device with subclasses Light, Thermostat, and SecurityCamera. Use overloaded constructors for devices with default vs. custom settings (e.g., brightness, temperature). Overload a turnOn() method for immediate activation vs. scheduled activation (turnOn(int delaySeconds)). Override an executeCommand() method per device type. Define a custom exception DeviceOfflineException. Implement each device as running on its own thread simulating periodic status updates, with a central controller thread issuing commands. Evaluate: Justify your design choice between using Runnable vs. extending Thread for the devices. Evaluate the risk of deadlock if the controller thread and a device thread both need to lock shared state (e.g., a device status log) — propose a strategy to prevent it and defend why it works.

Your Code

```java
import java.util.Scanner;
public class SmartHome {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String name = sc.nextLine();
        int brightness = sc.nextInt();
        Device light = new Light(name, brightness);
        new Thread(light).start();
        light.executeCommand();
        try {
            light.turnOn();
        } catch (DeviceOfflineException e) {
            System.out.println(e.getMessage());
        }
        int delay = sc.nextInt();
        light.turnOn(delay);
        sc.close();
    }
}
abstract class Device implements Runnable {
    String name;
    boolean online = true;
    Device(String name) {
        this.name = name;
    }
    Device(String name, boolean online) {
```

Input

```
Living Room Light
80
3
```

Output

```
Living Room Light light command
executed
Living Room Light status updated
Living Room Light turned ON
Living Room Light status updated
Living Room Light status updated
```

```java
        this.name = name;
        this.online = online;
    }
    abstract void executeCommand();
    void turnOn() throws DeviceOfflineException {
        if (!online)
            throw new DeviceOfflineException(name + " is offline");
        System.out.println(name + " turned ON");
    }
    void turnOn(int delay) {
        new Thread(() -> {
            try {
                Thread.sleep(delay * 1000L);
                turnOn();
            } catch (Exception e) {
                System.out.println(e.getMessage());
            }
        }).start();
    }
    public void run() {
        for (int i = 0; i < 3; i++) {
            System.out.println(name + " status updated");
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                break;
            }
        }
    }
```

```
    }
}
class Light extends Device {
    int brightness;
    Light(String name) {
        super(name);
        brightness = 50;
    }
    Light(String name, int brightness) {
        super(name);
        this.brightness = brightness;
    }
    void executeCommand() {
        System.out.println(name + " light command executed");
    }
}
class Thermostat extends Device {
    double temperature;
    Thermostat(String name) {
        super(name);
        temperature = 24;
    }
    Thermostat(String name, double temperature) {
        super(name);
        this.temperature = temperature;
    }
    void executeCommand() {
        System.out.println(name + " thermostat command executed");
```

```
    }
}
class SecurityCamera extends Device {
    SecurityCamera(String name) {
        super(name);
    }
    void executeCommand() {
        System.out.println(name + " camera command executed");
    }
}
class DeviceOfflineException extends Exception {
    DeviceOfflineException(String msg) {
        super(msg);
    }
}
```

Submitted